Universidad Rafael Landivar

Facultad de Ingeniería

Estructura de Datos II

Catedrático: Ingeniero Pablo Pineda

Segundo Proyecto

Aplicación de libros digitales (Utilizando Archivos)

Diego Alessandro Lopez Higueros - 1187525

Jose Pablo Osorio Grajeda – 1336325

1 de noviembre de 2

Contenido

1. Introducción 3

1.1 Relación con el enunciado 3

2. Requisitos y estructura 4

Archivos del servidor 4

Datos locales del cliente 4

3. Instalación y configuración 5

4. Uso de la aplicación 5

4.1 Inicio de sesión y catálogo 5

4.2 Lectura online 6

4.3 Descarga Premium y Huffman 7

4.4 Lectura offline 8

4.5 Cuenta estándar y validación 9

5. Algoritmos utilizados 9

5.1 SHA-256 9

5.2 Diffie-Hellman y Triple DES 9

5.3 Huffman 10

6. Evidencia de comunicación entre computadoras 11

6.1 Canal cifrado con Diffie-Hellman y Triple DES 11

6.2 Acceso de varios clientes en paralelo 11

7. Pruebas funcionales 12

8. Arquitectura del sistema 12

8.1 Paquetes 12

8.2 Diagramas flujo 13

9. Limitaciones y consideraciones 20

10. Solución de problemas 21

11. Observaciones y conclusiones 21

Anexo A. Datos de prueba 21

1. **Introducción**

Biblioteca Digital es una aplicación de consola desarrollada en Java mediante Apache NetBeans (Java with Ant, Java Application). Permite autenticar usuarios, consultar un catálogo alojado en una carpeta compartida, leer libros cifrados y, en cuentas Premium, descargarlos comprimidos para su lectura posterior sin conexión.

El sistema integra SHA-256 para la verificación de contraseñas, Diffie-Hellman para obtener secretos compartidos, Triple DES (3DES) para cifrar y descifrar y Huffman para comprimir los archivos descargados. La cola de prioridad que usa Huffman está implementada con clases propias (`ColaPrioridad` y `NodoHuffman`).

1.1 Relación con el enunciado

La siguiente tabla resume cómo cada requisito del enunciado se atiende en la solución y dónde se evidencia en este manual.

Requisito del enunciado	Cómo se cumple	Sección
Autenticar con hash contra el archivo de usuarios	Se calcula SHA-256 de la contraseña y se compara con el hash de *usuarios.txt*.	4.1, 5.1
Catálogo y libros cifrados en el servidor	*catalogo.txt* y la carpeta *libros/* se leen desde la carpeta compartida (ruta UNC en la configuración).	2, 3, 4.1
Lectura en línea (DH + 3DES, texto en consola)	Se copia el libro cifrado al cliente, se obtiene el secreto DH, se descifra con 3DES y se imprime.	4.2, 5.2
Descarga Premium (DH, 3DES y Huffman)	Se descifra, se comprime con Huffman, se verifica la integridad y se guarda un archivo de texto por usuario.	4.3, 5.3
Lectura local solo para Premium	Se descomprime el archivo descargado y se muestra el texto; no requiere el servidor.	4.4
Cuenta estándar: solo lectura en línea	El menú no ofrece descarga ni lectura local.	4.5
La copia cifrada permanece en el cliente	Queda en *data/cache_cifrada/* después de leer o descargar.	4.2
El archivo viaja por la aplicación (sin copy-paste)	La copia la realiza el programa mediante *ArchivoService*, por bloques.	4.2
Varios clientes en paralelo	Cada cliente lee los archivos del servidor por la carpeta compartida y escribe solo en su equipo.	6.2
Estructuras propias (sin librerías de Java)	Cola de prioridad y árbol de Huffman con clases y arreglos propios.	5.3, 8

2. Requisitos y estructura

Requisitos: Java JDK compatible con el proyecto, Apache NetBeans con soporte Ant, dos computadoras Windows conectadas a la misma red local para las pruebas distribuidas y acceso a una carpeta compartida SMB con permisos de lectura.

Proyecto: SegundoProyecto_ED2, con los paquetes *app, model, service, crypto, compression, storage* y *servidor* (descritos en la sección 8). La clase principal del cliente es `app.Main`; el servidor de demostración criptográfica se inicia mediante `servidor.ServidorDH`.

Archivos del servidor

La carpeta C:\BibliotecaServidor del servidor contiene los recursos que consumen los clientes:

Recurso	Formato y contenido
usuarios.txt	Una línea por usuario: *usuario*|*hash*|*premium*. El hash es SHA-256 en hexadecimal y *premium* es true o false.
catalogo.txt	Una línea por libro: *ISBN*|*título*.
libros/	Un archivo por libro, nombrado con el ISBN (por ejemplo 978001.txt). Contiene el IV de 8 bytes seguido del texto cifrado con 3DES; nunca el texto original.

El paquete de entrega incluye una copia de demostración de esos recursos.

Datos locales del cliente

Las rutas *data/...* son relativas al directorio de trabajo, que en NetBeans es la carpeta del proyecto cliente.

Ruta local	Función
data/configuracion.txt	Una sola línea con la ruta UNC de la carpeta compartida.
data/cache_cifrada/	Copias cifradas de los libros trasladados desde el servidor (permanecen disponibles).
data/descargas/<usuario>/	Libros descargados y comprimidos con Huffman, separados por usuario.
data/accesos_offline/	Autorización local (*usuario.properties*) que habilita la lectura sin conexión de cuentas Premium.

3. Instalación y configuración

1. Copiar la carpeta BibliotecaServidor (con *usuarios.txt*, *catalogo.txt* y *libros/*) a C:\BibliotecaServidor en la computadora servidor.
2. Compartir esa carpeta en Windows con el nombre BibliotecaServidor, habilitando lectura para la cuenta de acceso utilizada por los clientes.
3. En la computadora cliente, abrir *data/configuracion.txt* y escribir en una sola línea la ruta de red, con el formato *IP_DEL_SERVIDOR**BibliotecaServidor*; por ejemplo `\\192.168.1.100\BibliotecaServidor`.
   La dirección debe sustituirse por la IP real del servidor.
4. Abrir el proyecto en NetBeans y ejecutar `app.Main`. Para la prueba de comunicación DH, ejecutar también `servidor.ServidorDH` en la computadora servidor y `crypto.PruebaConexionDH` en la cliente.

Notas:

- El servidor DH utiliza el puerto TCP 5050 y, en la versión documentada, consulta el catálogo en la ruta local *C:\BibliotecaServidor*; por eso debe ejecutarse en la computadora servidor. Debe permitirse el tráfico del puerto en el firewall para la demostración.
- La clase *ClienteDH* obtiene la dirección del servidor a partir de la ruta UNC de *configuracion.txt* (el texto entre \\ y la siguiente \), por lo que la ruta debe respetar el formato indicado.
- Si la aplicación se ejecuta desde el archivo *dist/SegundoProyecto_ED2.jar* en lugar de NetBeans, debe lanzarse desde la carpeta del proyecto cliente para que se encuentre la carpeta *data/*.

4. Uso de la aplicación

Los diagramas de flujo de cada proceso descrito en esta sección se encuentran en la sección 8.3.

4.1 Inicio de sesión y catálogo

Al iniciar se muestra el menú general con las opciones de inicio de sesión online, lectura de libros descargados y salida. La autenticación compara el hash SHA-256 de la contraseña introducida con el registro del servidor. Tras una autenticación válida se identifica el tipo de cuenta.

El menú principal numera sus opciones como 1 (Explorar catálogo), 2 (Lectura local), 3 (Cerrar sesión) y 4 (Regresar al inicio). La opción 2 se muestra únicamente a las cuentas Premium, por lo que una cuenta estándar ve las opciones 1, 3 y 4 (Figura 5). Las opciones 3 y 4 finalizan la sesión actual y regresan al menú de inicio, desde el cual se puede iniciar otra sesión o salir.

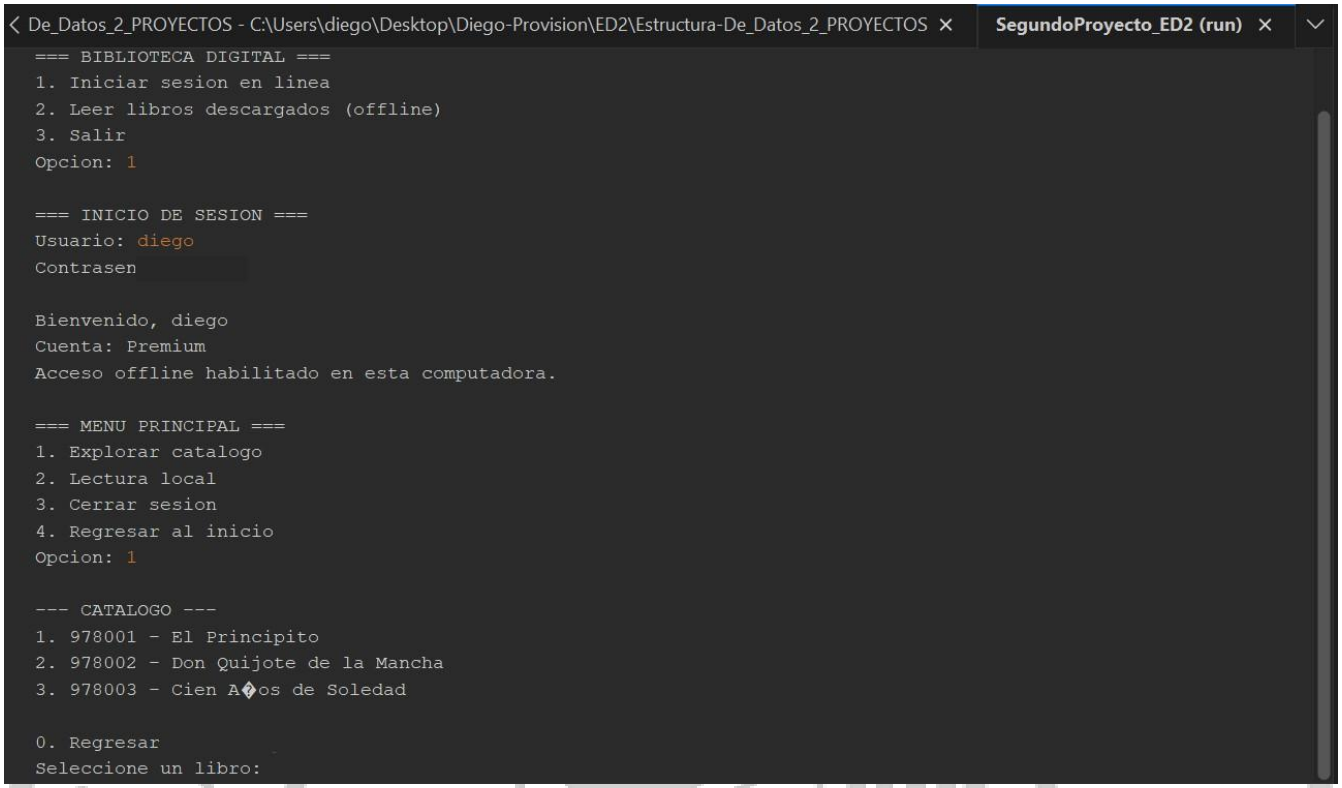


Figura 1. Menú principal, inicio de sesión Premium y catálogo de libros.

4.2 Lectura online

Al seleccionar un libro se transfiere primero una copia cifrada a la caché local, *data/cache_cifrada/<ISBN>.txt*, y esa copia permanece disponible en el cliente después de la lectura. La transferencia la ejecuta la propia aplicación (clase *ArchivoService*): lee el archivo del servidor por bloques de 4096 bytes, lo escribe en un archivo temporal y lo renombra al terminar, sin copiar y pegar entre carpetas.

Después, la lectura obtiene el secreto predeterminado de Diffie-Hellman utilizado para estos archivos, descifra el contenido mediante Triple DES y lo muestra en la consola. Las cuentas estándar inician esta lectura de inmediato al elegir el libro; las Premium eligen entre leer en línea o descargar.

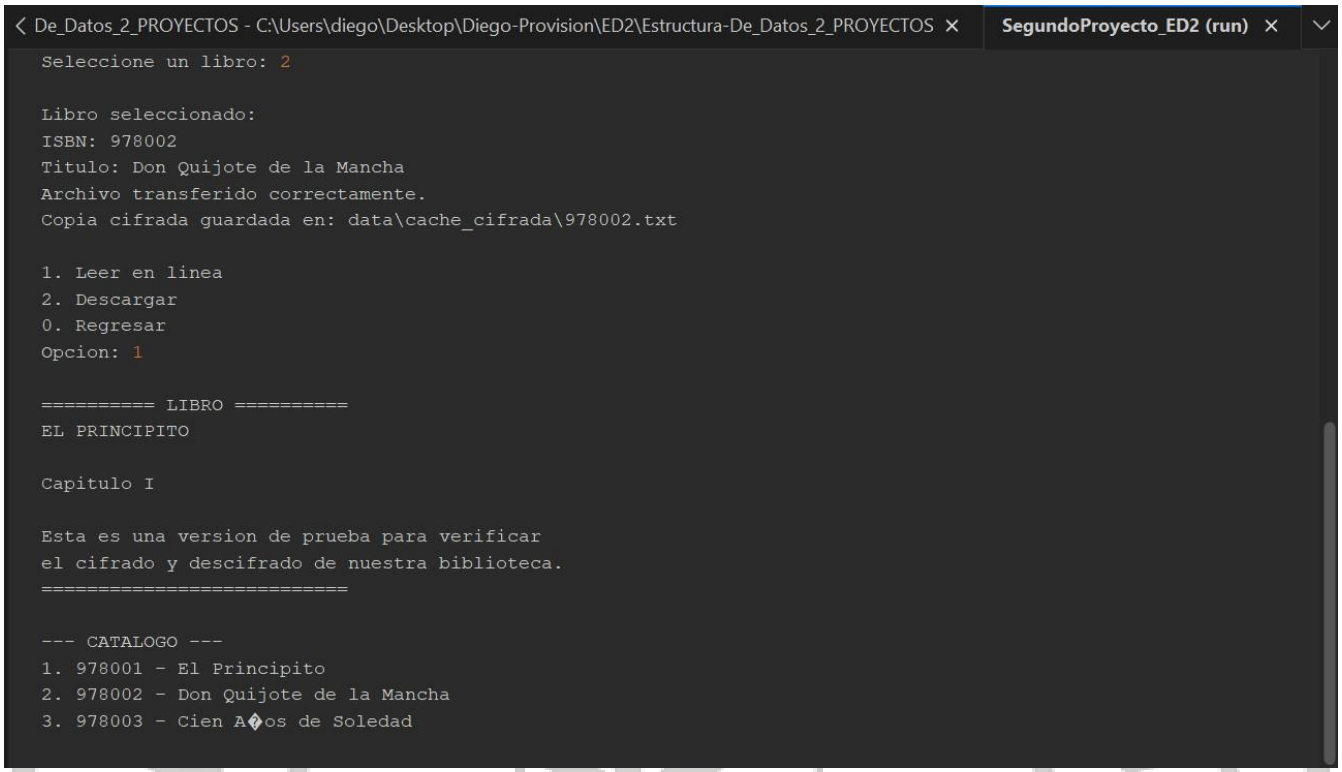


Figura 2. Selección de libro, copia cifrada y lectura en línea.

4.3 Descarga Premium y Huffman

Las cuentas Premium disponen de la opción de descarga. El proceso es el siguiente:

1. Se traslada la copia cifrada a *data/cache_cifrada/* (igual que en la lectura).
2. Se obtiene el secreto con Diffie-Hellman y se descifra el libro con 3DES.
3. Se comprime el texto con Huffman en un archivo temporal dentro de *data/descargas/<usuario>/*.
4. Se descomprime ese archivo temporal y se verifica que el texto recuperado sea idéntico al original.
5. Solo si la verificación es correcta, el temporal reemplaza a *<ISBN>.txt*; así una descarga anterior no se pierde si algo falla.
6. Se muestran el tamaño original, el tamaño guardado y el porcentaje de reducción.

```
Seleccione un libro: 2

Libro seleccionado:
ISBN: 978002
Titulo: Don Quijote de la Mancha
Archivo transferido correctamente.
Copia cifrada guardada en: data\cache_cifrada\978002.txt

1. Leer en linea
2. Descargar
0. Regresar
Opcion: 2
Libro descargado correctamente.
Integridad Huffman verificada.
Tama□o original: 123 bytes
Tama□o guardado: 431 bytes
Reduccion de tama□o: -250.41%

--- CATALOGO ---
1. 978001 - El Principito
2. 978002 - Don Quijote de la Mancha
3. 978003 - Cien A□os de Soledad

0. Regresar
Seleccione un libro:
```

Figura 3. Descarga y verificación de integridad Huffman.

Interpretación del tamaño: en la prueba mostrada, el archivo de 123 bytes pasa a 431 bytes. El tamaño aumenta porque el texto de demostración es muy breve y el formato comprimido incorpora diccionario y metadatos. La integridad se verifica correctamente; no se presenta este resultado como una reducción efectiva del tamaño. La sección 5.3 detalla, con este mismo archivo, de qué se compone el tamaño final.

4.4 Lectura offline

Una cuenta Premium que haya iniciado sesión online previamente en esa computadora puede autenticarse de manera local, seleccionar uno de sus libros descargados, descomprimirlo y leerlo sin acceder al servidor. Al iniciar sesión online como Premium, la aplicación guarda en *data/accesos_offline/<usuario>.properties* el nombre de usuario, el hash SHA-256 de la contraseña (no la contraseña) y la marca Premium; la validación offline compara contra ese archivo. Las descargas se mantienen separadas por usuario.

```
=== LECTURA OFFLINE PREMIUM ===
Usuario: pablo
Contrasena
Acceso offline Premium autorizado.

=== LIBROS DESCARGADOS ===
1. 978003.txt
0. Regresar
Seleccione un libro: 1

========== LIBRO ==========
EL PRINCIPITO

Capitulo I

Esta es una version de prueba para verificar
el cifrado y descifrado de nuestra biblioteca.
===========================
```

Figura 4. Autorización offline y lectura de un archivo descargado.

4.5 Cuenta estándar y validación

Las cuentas estándar pueden consultar y leer libros online, pero no tienen habilitada la descarga Premium ni la lectura local, y no generan autorización offline. Las credenciales incorrectas son rechazadas.



Figura 5. Menú de cuenta estándar, sin opción de lectura local.



Figura 6. Rechazo de credenciales incorrectas.

5. Algoritmos utilizados

5.1 SHA-256

El módulo de autenticación (*HashService*) calcula el resumen SHA-256 de la contraseña proporcionada, lo expresa en hexadecimal (64 caracteres) y lo compara con el hash registrado en *usuarios.txt*. La contraseña no se guarda en texto plano dentro del archivo de usuarios. Usuarios de ejemplo en el Anexo A.

5.2 Diffie-Hellman y Triple DES

Lectura y descarga de libros. El enunciado establece un intercambio Diffie-Hellman con valores predeterminados. La clase *DiffieHellman* usa el primo p = 23 y el generador g = 5, con los valores privados predeterminados del cliente (6) y del servidor (15):

Paso	Cálculo	Resultado
Valor público del cliente	$5^6 \bmod 23$	8
Valor público del servidor	$5^{15} \bmod 23$	19
Secreto calculado por el cliente	$19^6 \bmod 23$	2
Secreto calculado por el servidor	$8^{15} \bmod 23$	2

Ambos extremos obtienen el mismo secreto y el programa verifica que coincidan. Los archivos de libros se cifran y descifran con 3DES así:

- La clave de 24 bytes se obtiene aplicando SHA-256 a la representación decimal del secreto y tomando los primeros 24 bytes.
- Algoritmo *DESede/CBC/PKCS5Padding* con un IV aleatorio de 8 bytes que se guarda al inicio del archivo cifrado. Por ello el texto de 123 bytes ocupa 136 bytes cifrado (8 de IV + 128 con relleno).

Canal de consulta por sockets. Por separado, la prueba de comunicación (*ClienteDH* y *ServidorDH*) negocia un intercambio Diffie-Hellman real con claves de 2048 bits (API estándar de Java), deriva la clave 3DES con SHA-256 del secreto y cifra cada mensaje con un IV aleatorio, codificando el resultado en Base64.

5.3 Huffman

El algoritmo construye códigos de longitud variable según las frecuencias de los caracteres. El programa incluye su propia estructura de cola de prioridad (*ColaPrioridad*, un arreglo ordenado por frecuencia) y nodos propios (*NodoHuffman*) para construir el árbol: se extraen los dos nodos de menor frecuencia, se crea un padre con la suma y se reinserta, hasta quedar un solo nodo. En el código, el hijo izquierdo aporta un 0 y el derecho un 1. Si el texto tiene un único carácter distinto, se le asigna el código 0. La integridad se comprueba durante la descarga descomprimiendo y comparando con el texto original.

Formato del archivo comprimido. El resultado es un archivo de texto con cuatro partes:

Parte	Contenido
1. Cantidad de símbolos	Número de caracteres distintos del texto.
2. Tabla de códigos	Una línea por símbolo: *código_Unicode*|*bits*. Por ejemplo, *97*|*000* representa la letra «a» con el código 000.
3. Cantidad de bits	Longitud exacta del mensaje codificado, para ignorar el relleno del último byte.
4. Datos	Los bits agrupados en bytes y codificados en Base64.

Ejemplo real. El archivo descargado de 431 bytes de la Figura 3 comienza así (se omiten las líneas intermedias de la tabla):

```
30
10|11001
13|11100
32|011
...
121|010010
558 svu3FLAnK2WPDzPMphC0qbzlzPM2foHv3U4d2x9fHfdaIrpA0CA7thSr...
```

El texto de 123 caracteres usa 30 símbolos distintos y se codifica en 558 bits (70 bytes), es decir, el contenido por sí solo ocupa alrededor de 43 % menos que el original. Sin embargo, el archivo incluye además las 30 líneas de la tabla y el contenido en Base64 (que aumenta cerca de un tercio el tamaño de los datos), por lo que el total es 431 bytes. En textos largos el costo de la tabla (una línea por carácter distinto) se reparte entre muchos más caracteres, de modo que la reducción total se acerca a la del contenido codificado.

6. Evidencia de comunicación entre computadoras

6.1 Canal cifrado con Diffie-Hellman y Triple DES

En la computadora servidor se ejecutó ServidorDH, que quedó a la escucha en el puerto 5050. En el cliente se ejecutó PruebaConexionDH: se envió una solicitud cifrada, se obtuvo una respuesta legible del servidor y se mostró el mensaje de prueba exitosa.

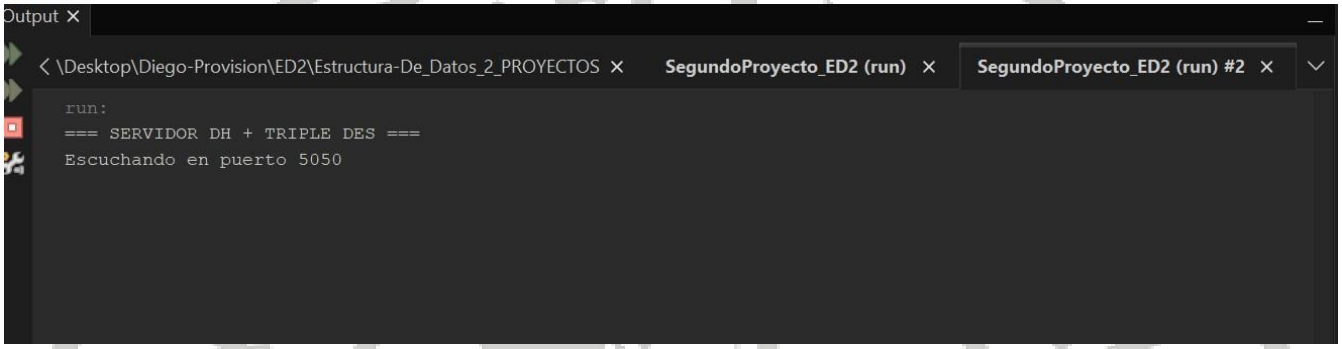


Figura 7. Servidor DH escuchando conexiones TCP en el puerto 5050.



Figura 8. Cliente: mensaje cifrado, respuesta del servidor y ejecución exitosa.

La respuesta observada indica: *Servidor activo. Registros del catálogo: 3.* Esta prueba demuestra el canal de consulta cifrado; el acceso a los archivos de libros se realiza por la carpeta compartida SMB y no a través de este socket. El servidor atiende las conexiones de una en una y solo reconoce la solicitud *ESTADO*.

6.2 Acceso de varios clientes en paralelo

El enunciado pide que varios usuarios puedan usar a la vez los archivos de usuarios, catálogo y la carpeta de libros. En esta solución:

- Cada cliente ejecuta su propia instancia de la aplicación y lee *usuarios.txt*, *catalogo.txt* y *libros/* directamente desde la carpeta compartida SMB.
- Los clientes solo leen la carpeta del servidor; nunca la modifican, por lo que las lecturas simultáneas no generan conflictos de escritura.
- Todo lo que cada cliente escribe (*cache_cifrada*, *descargas* y *accesos_offline*) queda en su propio equipo.
- El socket del puerto 5050 es independiente del acceso a los libros y solo se usa en la demostración del canal cifrado.

7. Pruebas funcionales

Prueba	Resultado reportado	Evidencia
Autenticación Premium y estándar	Satisfactoria	Figuras 1 y 5
Consulta de catálogo y lectura online	Satisfactoria	Figuras 1 y 2
Descarga Premium e integridad Huffman	Satisfactoria	Figura 3
Lectura offline por usuario	Satisfactoria	Figura 4
Rechazo de credenciales incorrectas	Satisfactoria	Figura 6
Prueba	Resultado reportado	Evidencia
Clientes simultáneos en red local	Satisfactoria	Sin captura (ver nota)
Consulta cifrada DH + Triple DES	Satisfactoria	Figuras 7 y 8

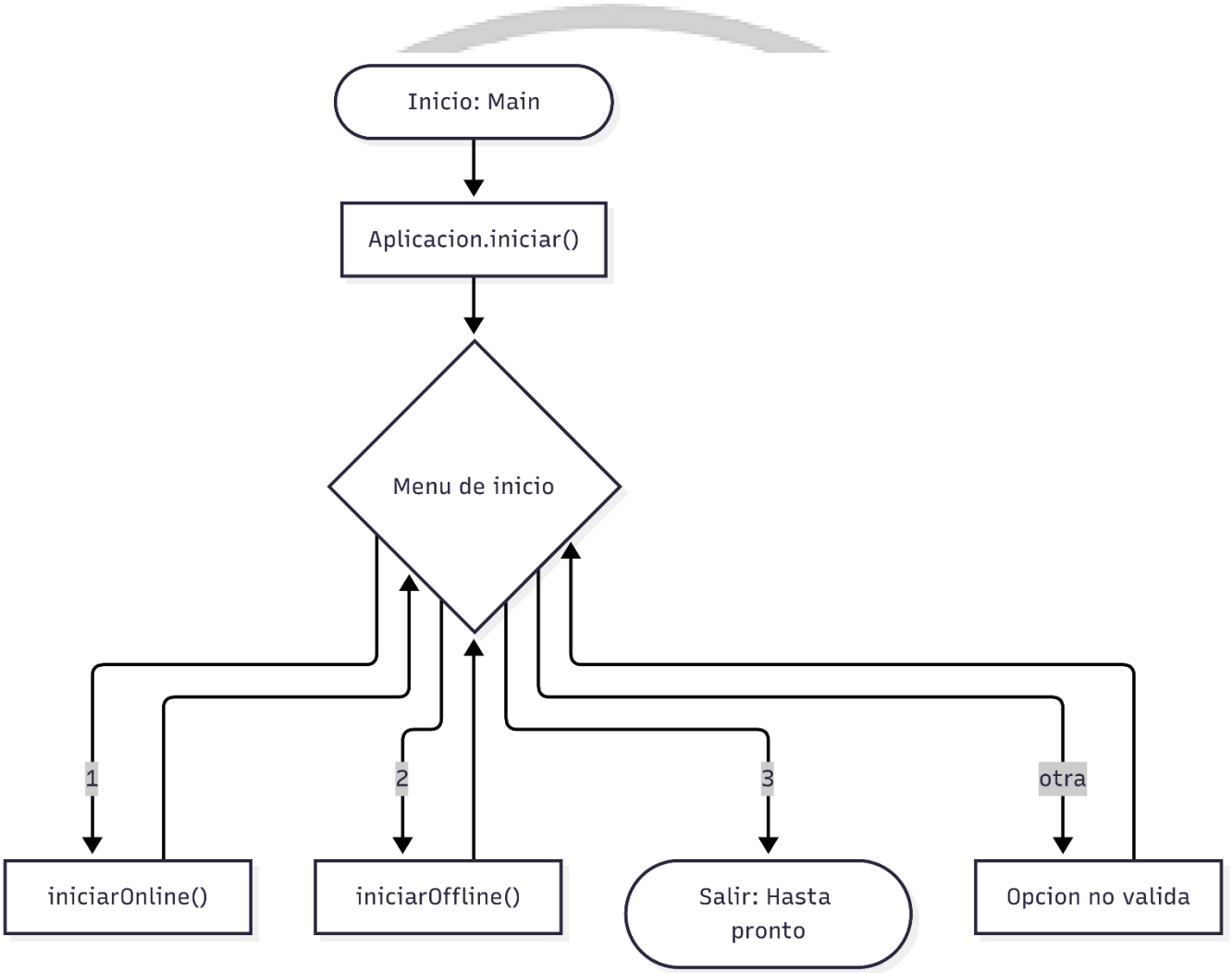
Los resultados se basan en las capturas suministradas y en las pruebas funcionales reportadas por los desarrolladores. La prueba de simultaneidad se reportó como satisfactoria, aunque no se adjuntó una captura específica de ambas sesiones en paralelo.

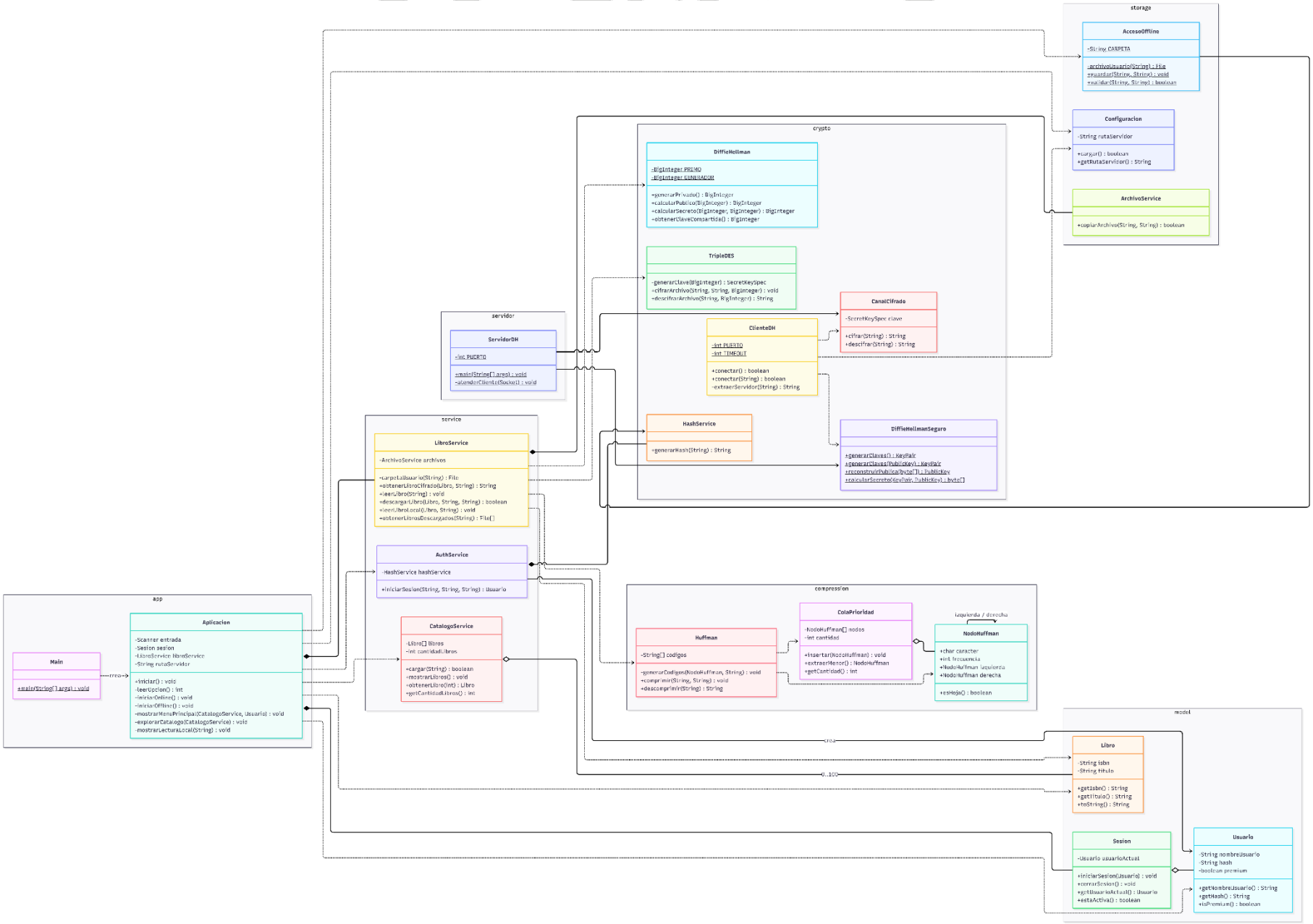
8. Arquitectura del sistema

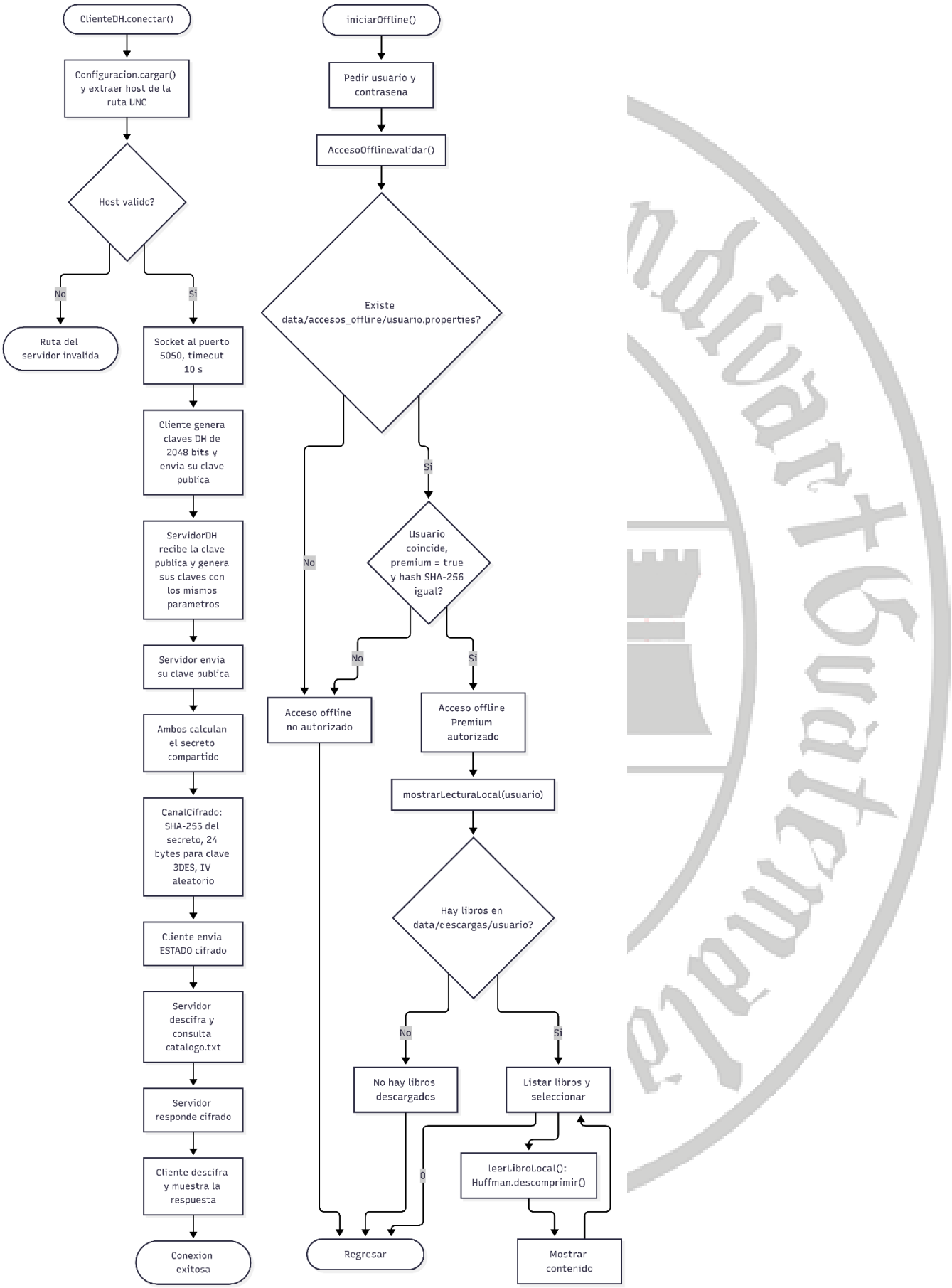
8.1 Paquetes

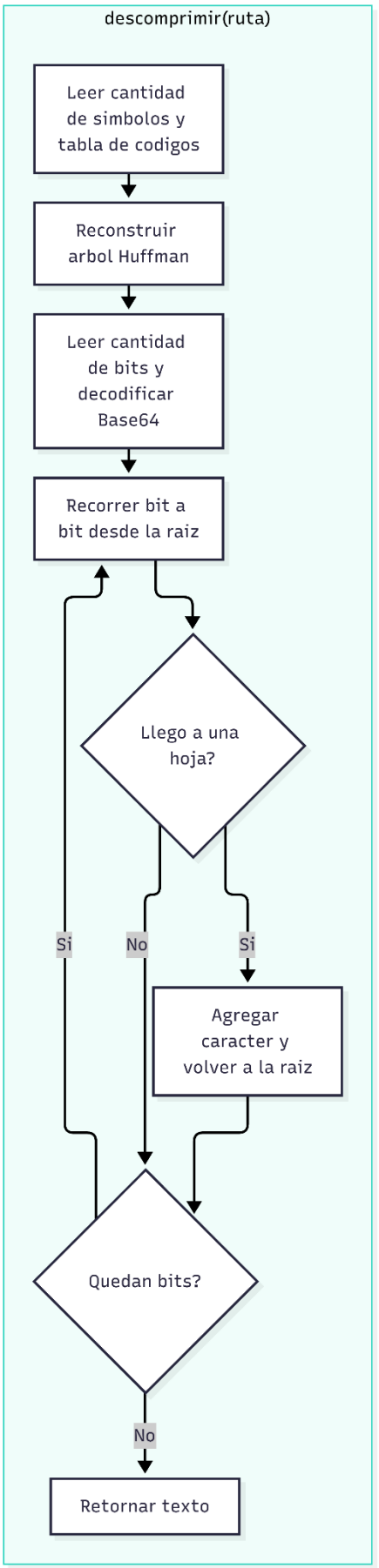
Paquete	Clases	Responsabilidad
app	Main, Aplicacion	Punto de entrada, menús y flujo de la aplicación.
model	Libro, Usuario, Sesion	Datos del libro, del usuario autenticado y de la sesión activa.
service	AuthService, CatalogoService, LibroService	Autenticación, carga del catálogo, lectura, descarga y lectura local de libros.
storage	Configuracion, ArchivoService, AccesoOffline	Configuración, traslado de archivos y autorización offline.
compression	Huffman, ColaPrioridad, NodoHuffman	Compresión y descompresión Huffman con estructuras propias.
crypto	HashService, DiffieHellman, TripleDES, DiffieHellmanSeguro, CanalCifrado, ClienteDH	SHA-256, intercambio de claves, cifrado y canal cifrado cliente-servidor.
servidor	ServidorDH	Servidor de demostración del canal cifrado (puerto 5050).

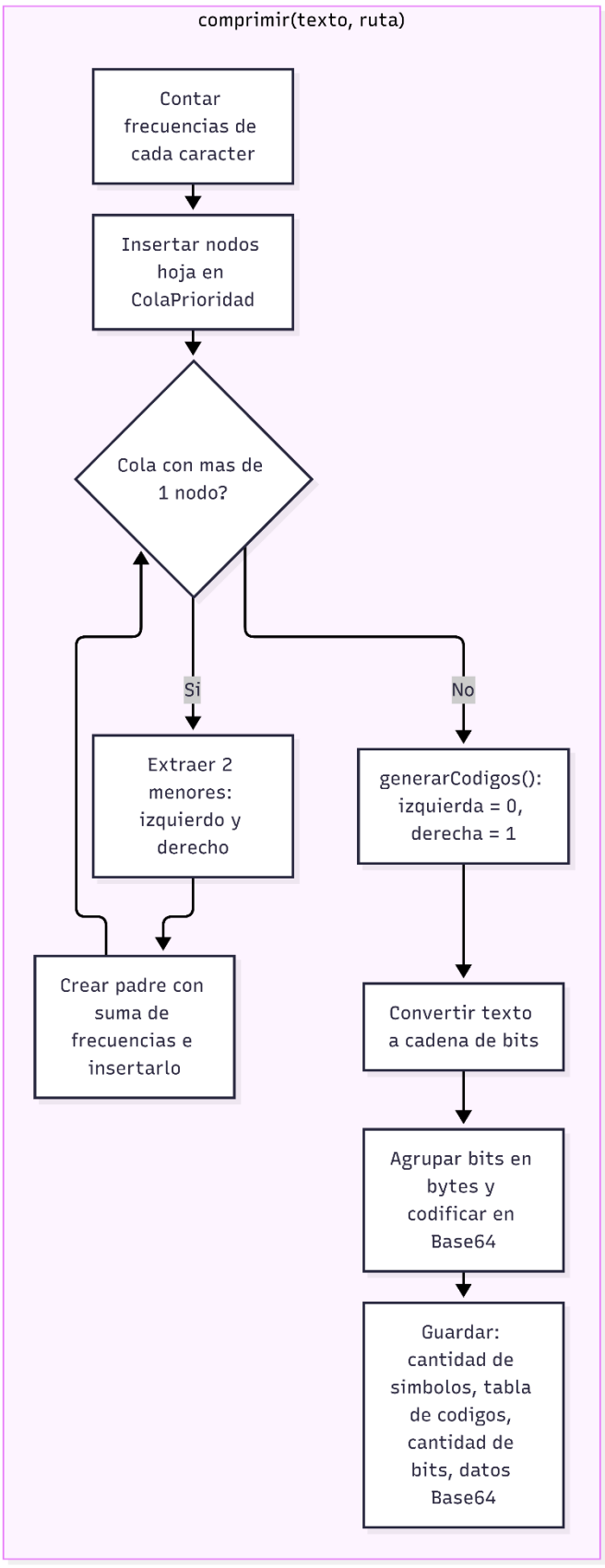
8.2 Diagramas flujo

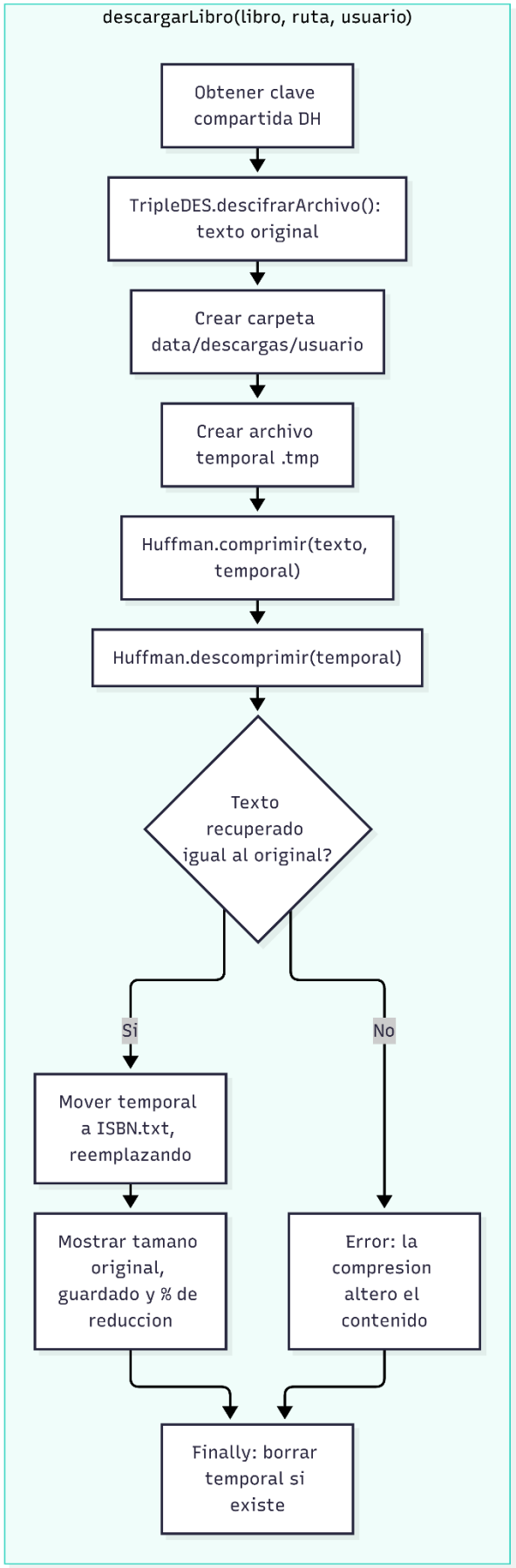


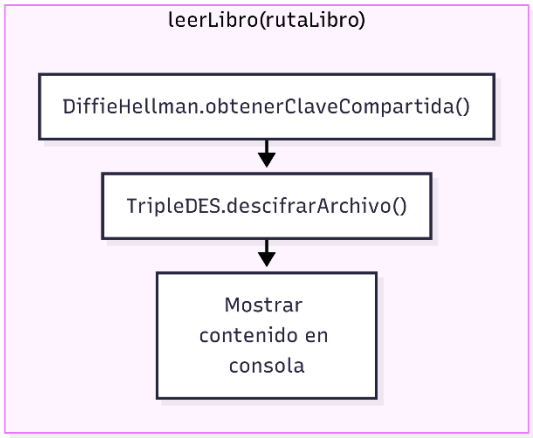

storage
AccesoOffline
-String CARPETA
-archivoUsuario(String) : File
+guardar(String, String) : void
+validar(String, String) : boolean
Configuracion
-String rutaServidor
+cargar() : boolean
+getRutaServidor() : String
ArchivoService
+copiarArchivo(String, String) : boolean
crypto
DiffieHellman
-BigInteger PRIMO
-BigInteger GENERADOR
+generarPrivado() : BigInteger
+calcularPublico(BigInteger) : BigInteger
+calcularSecreto(BigInteger, BigInteger) : BigInteger
+obtenerClaveCompartida() : BigInteger
TripleDES
-generarClave(BigInteger) : SecretKeySpec
+cifrarArchivo(String, String, BigInteger) : void
+descifrarArchivo(String, BigInteger) : String
CanalCifrado
-SecretKeySpec clave
+cifrar(String) : String
+descifrar(String) : String
ClienteDH
-int PUERTO
-int TIMEOUT
+conectar() : boolean
+conectar(String) : boolean
-extraerServidor(String) : String
HashService
+generarHash(String) : String
DiffieHellmanSeguro
+generarClaves() : KeyPair
+generarClaves(PublicKey) : KeyPair
+reconstruirPublica(byte[]) : PublicKey
+calcularSecreto(KeyPair, PublicKey) : byte[]
servidor
ServidorDH
-int PUERTO
+main(String[] args) : void
-atenderCliente(Socket) : void
service
LibroService
-ArchivoService archivos
-carpetaUsuario(String) : File
+obtenerLibroCifrado(Libro, String) : String
+leerLibro(String) : void
+descargarLibro(Libro, String, String) : boolean
+leerLibroLocal(Libro, String) : void
+obtenerLibrosDescargados(String) : File[]
AuthService
-HashService hashService
+iniciarSesion(String, String, String) : Usuario
CatalogoService
-Libro[] libros
-int cantidadLibros
+cargar(String) : boolean
+mostrarLibros() : void
+obtenerLibro(int) : Libro
+getCantidadLibros() : int
compression
Huffman
-String[] codigos
-generarCodigos(NodoHuffman, String) : void
+comprimir(String, String) : void
+descomprimir(String) : String
ColaPrioridad
-NodoHuffman[] nodos
-int cantidad
+insertar(NodoHuffman) : void
+extraerMenor() : NodoHuffman
+getCantidad() : int
izquierda / derecha
NodoHuffman
+char caracter
+int frecuencia
+NodoHuffman izquierda
+NodoHuffman derecha
+esHoja() : boolean
app
Main
+main(String[] args) : void
crea
Aplicacion
-Scanner entrada
-Sesion sesion
-LibroService libroService
-String rutaServidor
+iniciar() : void
-leerOpcion() : int
-iniciaOnline() : void
-iniciarOffline() : void
-mostrarMenuPrincipal(CatalogoService, Usuario) : void
-explorarCatalogo(CatalogoService) : void
-mostrarLecturaLocal(String) : void
crea
0..100
model
Libro
-String isbn
-String titulo
+getIsbn() : String
+getTitulo() : String
+toString() : String
Sesion
-Usuario usuarioActual
+iniciarSesion(Usuario) : void
+cerrarSesion() : void
+getUsuarioActual() : Usuario
+estaActiva() : boolean
Usuario
-String nombreUsuario
-String hash
-boolean premium
+getNombreUsuario() : String
+getHash() : String
+isPremium() : boolean

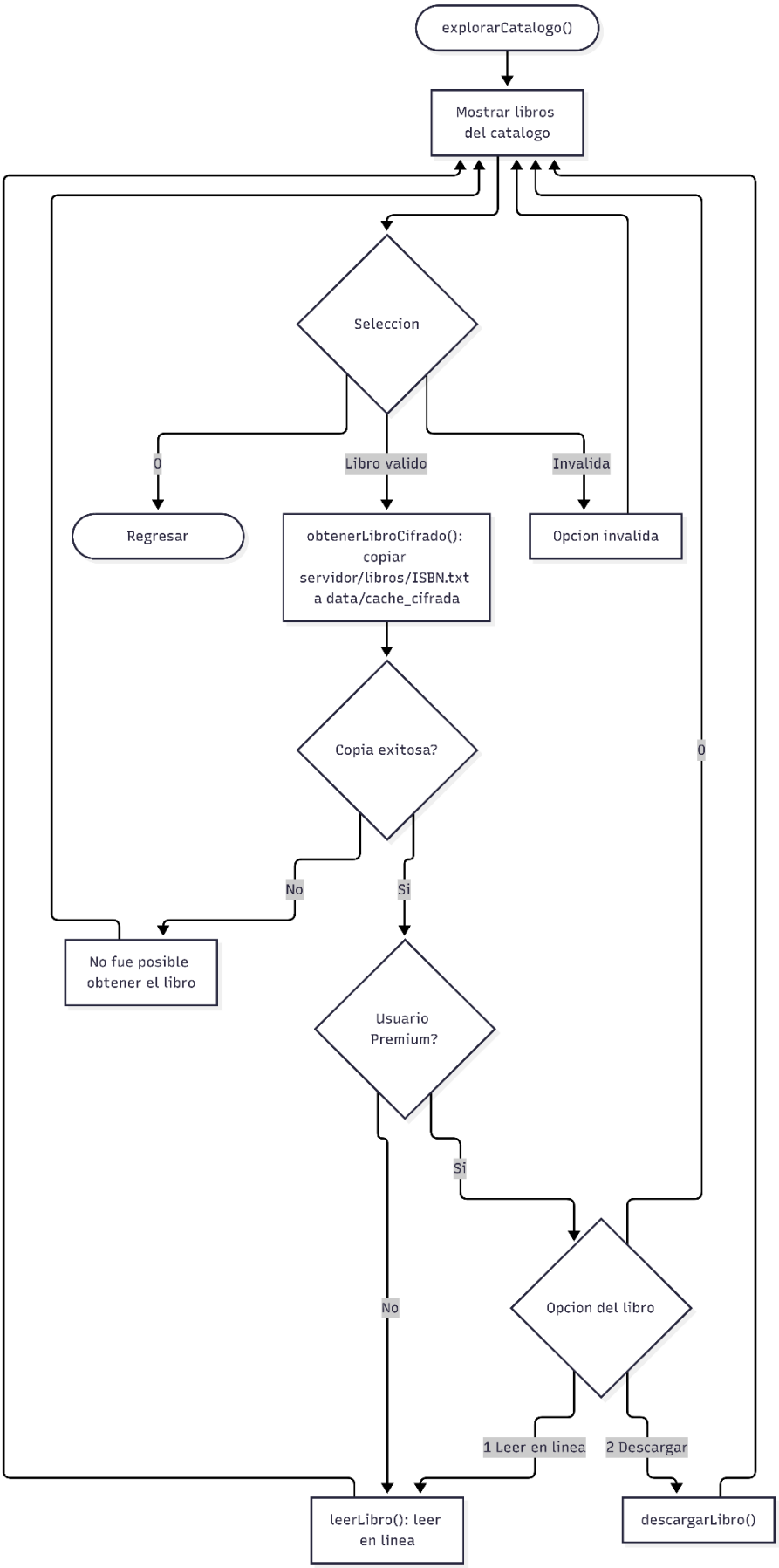

ClienteDH.conectar()
Configuracion.cargar() y extraer host de la ruta UNC
Host valido?
No
Si
Ruta del servidor invalida
Socket al puerto 5050, timeout 10 s
Cliente genera claves DH de 2048 bits y envia su clave publica
ServidorDH recibe la clave publica y genera sus claves con los mismos parametros
Servidor envia su clave publica
Ambos calculan el secreto compartido
CanalCifrado: SHA-256 del secreto, 24 bytes para clave 3DES, IV aleatorio
Cliente envia ESTADO cifrado
Servidor descifra y consulta catalogo.txt
Servidor responde cifrado
Cliente descifra y muestra la respuesta
Conexion exitosa
iniciarOffline()
Pedir usuario y contrasena
AccesoOffline.validar()
Existe data/accesos_offline/usuario.properties?
No
Si
Usuario coincide, premium = true y hash SHA-256 igual?
No
Si
Acceso offline no autorizado
Acceso offline Premium autorizado
mostrarLecturaLocal(usuario)
Hay libros en data/descargas/usuario?
No
Si
No hay libros descargados
Listar libros y seleccionar
0
leerLibroLocal(): Huffman.descomprimir()
Regresar
Mostrar contenido

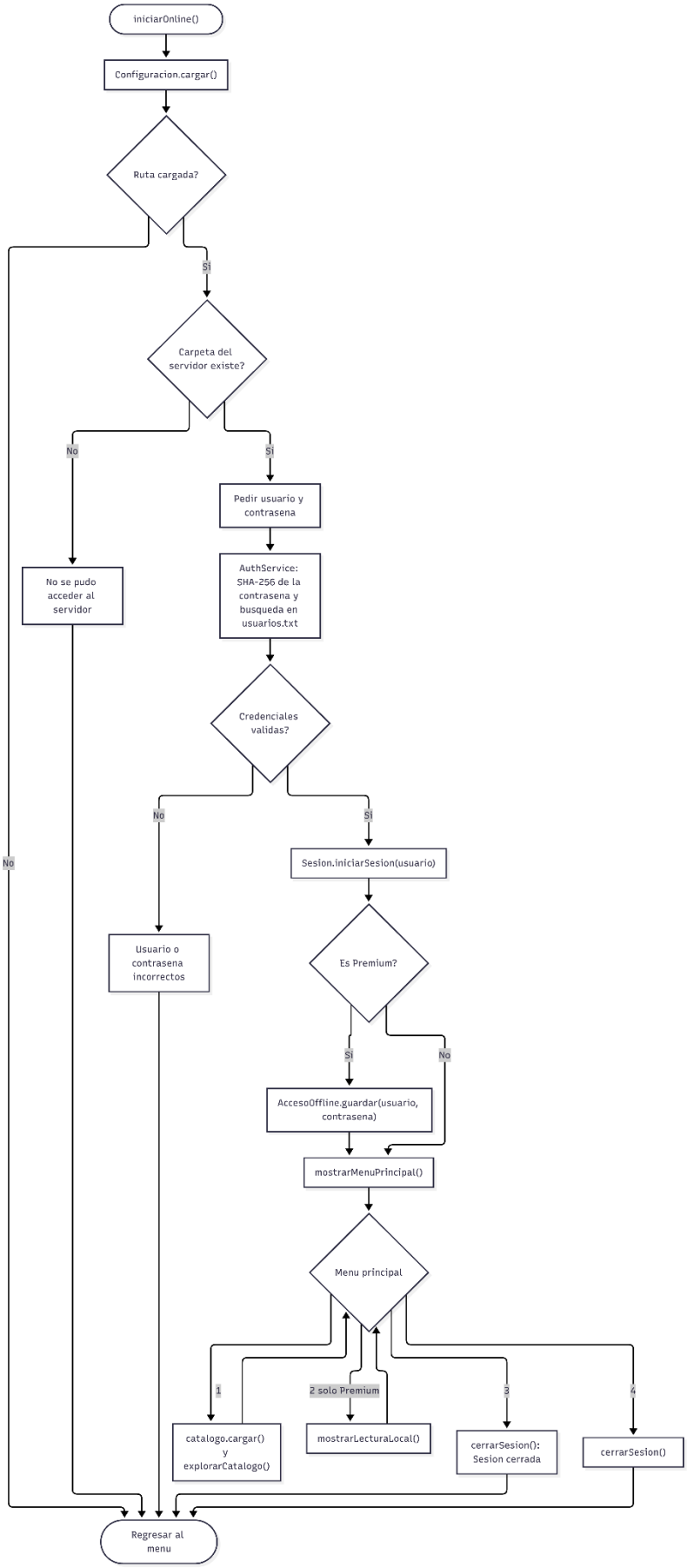

descomprimir(ruta)
Leer cantidad de simbolos y tabla de codigos
Reconstruir arbol Huffman
Leer cantidad de bits y decodificar Base64
Recorrer bit a bit desde la raiz
Llego a una hoja?
Si
No
Si
Agregar caracter y volver a la raiz
Quedan bits?
No
Retornar texto


comprimir(texto, ruta)
Contar frecuencias de cada caracter
Insertar nodos hoja en ColaPrioridad
Cola con mas de 1 nodo?
Si
No
Extraer 2 menores: izquierdo y derecho
Crear padre con suma de frecuencias e insertarlo
generarCodigos(): izquierda = 0, derecha = 1
Convertir texto a cadena de bits
Agrupar bits en bytes y codificar en Base64
Guardar: cantidad de simbolos, tabla de codigos, cantidad de bits, datos Base64


descargarLibro(libro, ruta, usuario)
Obtener clave compartida DH
TripleDES.descifrarArchivo(): texto original
Crear carpeta data/descargas/usuario
Crear archivo temporal .tmp
Huffman.comprimir(texto, temporal)
Huffman.descomprimir(temporal)
Texto recuperado igual al original?
Si
No
Mover temporal a ISBN.txt, reemplazando
Mostrar tamano original, guardado y % de reduccion
Error: la compresion altero el contenido
Finally: borrar temporal si existe


leerLibro(rutaLibro)
DiffieHellman.obtenerClaveCompartida()
TripleDES.descifrarArchivo()
Mostrar contenido en consola


explorarCatalogo()
Mostrar libros del catalogo
Seleccion
0
Libro valido
Invalida
Regresar
obtenerLibroCifrado(): copiar servidor/libros/ISBN.txt a data/cache_cifrada
Opcion invalida
Copia exitosa?
0
No
Si
No fue posible obtener el libro
Usuario Premium?
Si
No
Opcion del libro
1 Leer en linea
2 Descargar
leerLibro(): leer en linea
descargarLibro()


iniciarOnline()
Configuracion.cargar()
Ruta cargada?
Si
No
Carpeta del servidor existe?
No
Si
No se pudo acceder al servidor
Pedir usuario y contrasena
AuthService: SHA-256 de la contrasena y busqueda en usuarios.txt
Credenciales validas?
No
Si
Usuario o contrasena incorrectos
Sesion.iniciarSesion(usuario)
Es Premium?
Si
No
AccesoOffline.guardar(usuario, contrasena)
mostrarMenuPrincipal()
Menu principal
1
2 solo Premium
3
4
catalogo.cargar() y explorarCatalogo()
mostrarLecturaLocal()
cerrarSesion(): Sesion cerrada
cerrarSesion()
Regresar al menu

9. Limitaciones y consideraciones

La implementación tiene fines académicos. Se documentan las siguientes decisiones y límites para evitar interpretaciones erróneas:

- Diffie-Hellman de los libros: usa los valores predeterminados exigidos por el enunciado (p = 23, g = 5, privados 6 y 15), por lo que el secreto siempre es 2. Es adecuado para demostrar el algoritmo, no para proteger información real. El canal de consulta por sockets sí usa claves de 2048 bits.
- Triple DES es el algoritmo simétrico solicitado; hoy se considera obsoleto frente a AES.
- SHA-256 sin sal: dos usuarios con la misma contraseña tienen el mismo hash (ocurre con los usuarios de prueba *diego* e *invitado*).
- Acceso offline: se guarda el hash de la contraseña, no la contraseña, en *data/accesos_offline/*.
- Nombres de usuario: solo se aceptan letras, números, guion y guion bajo, porque forman parte de rutas de archivo.
- Servidor DH: atiende un cliente a la vez, solo responde *ESTADO* y lee el catálogo en la ruta fija *C:\BibliotecaServidor\catalogo.txt*.
- Catálogo: admite hasta 100 libros (arreglo de tamaño fijo).
- Textos cortos y Huffman: pueden ocupar más espacio comprimidos que originales por el costo de la tabla (sección 5.3).
- Libros de demostración: los libros y sus contenidos son de prueba.

10. Solución de problemas

Mensaje o síntoma	Causa probable	Solución
Error al cargar la configuración	No existe *data/configuracion.txt* o la aplicación se ejecutó desde otra carpeta.	Ejecutar desde la carpeta del proyecto y verificar el archivo.
La ruta del servidor está vacía	El archivo de configuración no tiene texto en la primera línea.	Escribir la ruta UNC en la primera línea.
No se pudo acceder al servidor	Ruta UNC incorrecta, servidor apagado o sin permisos de lectura.	Abrir la ruta en el Explorador de Windows; revisar IP, nombre del recurso compartido y permisos.
Usuario o contraseña incorrectos	Credenciales erróneas o línea mal formada en *usuarios.txt*.	Revisar el Anexo A y el formato *usuario|hash|premium*.
No se pudo descifrar el libro	El archivo no fue cifrado con el secreto y el algoritmo esperados.	Regenerar los libros con `crypto.PruebaCifrado`.
Error de conexión cifrada	ServidorDH no está en ejecución, firewall bloquea el puerto 5050 o la ruta UNC no tiene el formato esperado.	Iniciar ServidorDH, permitir el puerto 5050 y revisar la configuración.
Acceso offline no autorizado	La cuenta no es Premium o nunca inició sesión online en esa computadora.	Iniciar sesión online con una cuenta Premium al menos una vez.
No hay libros descargados	El usuario aún no ha descargado libros.	Descargar un libro desde Explorar catálogo.

11. Observaciones y conclusiones

La aplicación permite distinguir permisos de usuario, acceder a libros cifrados, mantener descargas independientes y recuperar contenido sin conexión. La comunicación de prueba por sockets incorpora intercambio de claves y cifrado simétrico. La organización modular facilita el mantenimiento y la demostración de los algoritmos solicitados.

Para trasladar la solución a otra red es necesario actualizar la ruta UNC y verificar los permisos de la carpeta compartida. La implementación tiene fines académicos y utiliza datos y libros de demostración; sus limitaciones se detallan en la sección 9.

Anexo A. Datos de prueba

Usuarios incluidos en el *usuarios.txt* de demostración. El archivo almacena únicamente el hash SHA-256; la contraseña se muestra aquí solo para reproducir las pruebas.

Usuario	Contraseña de prueba	Tipo de cuenta	Habilita
diego	123456	Premium	Lectura online, descarga y lectura local/offline
pablo	123	Premium	Lectura online, descarga y lectura local/offline
invitado	123456	Estándar	Solo lectura online

Catálogo de demostración (*catalogo.txt*):

ISBN	Título
978001	El Principito
978002	Don Quijote de la Mancha
978003	Cien Años de Soledad